

Two fixed small models, one Lean compiler: does collaboration beat either model alone, and why not here?

Verified Mechanisms Research Engineer take-home. Jacob Belenkii, September 2026

Numbers marked [repo] come from committed files. The evidence branches `evidence/results-20260902`, `evidence/online-development-v1-stage3-2e157547`, and `evidence/run-data-*` hold every cell's `result.json`, `events.jsonl`, `transcript.json`, and provenance.

1. The answers

Two fixed models, Qwen 3.5 Flash and GPT-OSS 120B, each run a repair loop against a Lean 4 compiler on the same theorem; the question is whether letting them exchange anything beats running them side by side, and whether either beats running one alone. The brief asks two questions: does a two-model system beat either model alone, and where does one model fill the other's gaps. Three answers, each with its section.

Terms used throughout. A *cell* is one attempt at one problem by one condition. A *track* is one model's repair loop inside a cell; a *portfolio* is two tracks on the same problem, *silent* if they never exchange anything and *talking* if they do; a *packet* is what crosses between tracks. A candidate is *warm-accepted* when the agent's own Lean check compiles it, and a cell *passes* when the graders' Comparator (a fresh, pinned build that also checks the theorem statement was not altered) accepts the returned file. A *block* is one problem at one replication, attempted by every condition. The *band* is the set of problems solved sometimes but not always, where a difference between conditions can show; the *floor set* is the problems nothing solved.

The two-model system beats either model alone by about what any second track buys, and on this set a second Qwen track would buy nearly as much as GPT-OSS does. On this harness at one seed, Qwen's track solves 9/16. GPT-OSS's track solves 6/16, all of them problems Qwen also solves. The silent two-model portfolio solves 9/16 (§4). On the four band problems at six new seeds, the portfolio solves 19 of 24 blocks against Qwen's 16, GPT-OSS's 9, and their union of 17: the second track is worth about one band problem in ten, and a second Qwen track would have been worth nearly as much as a GPT-OSS one (§4.3). The portfolio returns whichever proof compiles first. The largest uplift in the project is harness engineering on a single track, not the second model: `p10_factorial_pow` is 0/2 under the supplied baseline agent and 14/14 under this engine in every arm (§5).

Qwen searches; GPT-OSS structures. Inside the portfolio, Qwen's track produced 53 of the 80 passing proofs, GPT-OSS's track 23, the tactic cascade 4. `p08`, `p09`, and `p10` are Qwen's. `p05` is GPT-OSS's or the cascade's. The rest are near-ties. GPT-OSS's failures leave a compiling skeleton with real proof content 40% of the time; Qwen's failures, 11% (§5).

Letting the tracks talk adds nothing detectable. Three exchange mechanisms ran on 84 + 56 matched cells. The silent controls passed 15/28 and 17/28; the talking arms 16/28, 16/28, and 16/28. Search curves sit within one cell of each other at every budget (§6). Packets were produced, delivered, and consumed. In the second mechanism they yielded ten Comparator-passing proofs, every one on a block the silent twin also solved. A four-arm powered study on the four band problems at six new seeds, 96 cells, gives the same answer with an interval: the talking portfolio solved 17 of 24 blocks, the silent one 19; talking minus silent is -0.08 (-0.25 to $+0.04$) on the band (§6.3). A counterfactual replay of every packet-exposed request, eight samples per variant, shows why at the request level: with the packet in the exact recorded context the recipient's next response compiles at the same rate as without it (-0.01 , interval -0.10 to $+0.10$ for progress packets; $+0.00$ for failed-candidate packets; §6.2b). The one real exception is a packet that is already a proof. The reason at the problem level is structural: exchange can only add on blocks where both tracks fail alone, and on this set those are the five floor problems, on which no partial structure crosses, plus seven band blocks at unlucky seeds, on which the talking arm scored 1 and the silent arm 2 (§6.4).

§2 describes what was built, §3 how it was measured, §4 to §6 the evidence for each answer in

turn.

2. What was built

One solver, several communication policies. Every condition is the same ~900-line engine with a strategy object that can only read observations and emit packets; it cannot touch budgets, scheduling, or verification. That is what makes the conditions comparable and the packet log interpretable.

The shared solver. A deterministic tactic cascade (`omega`, `norm_num`, `nlinarith`, `linarith`, `ring`, `aesop`, `simp_all`) is tried first on every hole; it closes only `p05_gcd_mersenne`, identically in every condition. Then each model runs an independent repair track from the pristine challenge: it sees the statement, its own current candidate, bounded compiler diagnostics, and a bounded memory of abandoned approaches, and restarts from scratch (at most twice) on a repeated candidate or error signature. Temperature 0.2, 12,000-token generations, explicit `reasoning.effort = medium` for both models, a shared per-cell seed, 25 calls per track [repo: `online-development-v1/METHODS.md`]. Medium equalizes the requested effort, not tokens or latency (Qwen ~3.6k reasoning tokens in 24 s against GPT-OSS ~10k in 208 s on the same request; \$7). Tracks run asynchronously, since in lockstep GPT-OSS's latency tail (medians 70–100 s, maxima 250–400 s) idles Qwen for most of a cell. The compiler, not text matching, decides candidate safety: a declaration-presence gate, then the pinned Comparator run fresh before any candidate may stop the search.

Family one: failed-candidate packets (pilot only, superseded). Up to 6,000 characters of the peer's latest non-compiling candidate plus diagnostics, with an instruction to evaluate critically. **C0**: no communication. **C1**: one reciprocal exchange after the first round in which both tracks fail. **C2**: an exchange after every such round.

Family two: compiler-grounded progress packets with a reserved call. On a Lean failure the harness salvages a compiling skeleton (the smallest failing span becomes sorry, re-checked in two seconds, kept only if it retains a proof-body line); skeletons are internal state, never returned. Replay over 100 archived failures yielded a useful skeleton 24% of the time, 18/45 for GPT-OSS and 6/55 for Qwen [repo: `STAGE0_RESULTS.md`]. **C0+reserve**: within-track salvage only; one Qwen call held back until GPT-OSS exhausts its calls or 120 s remain. **C1+fill-reserve**: the same, plus latest-wins packets carrying the peer's skeleton, residual goals, and verified helpers, with one instruction (keep the skeleton, fill the holes); a packet unlocks the reserved call. Same calls, budgets, timeouts, and reserve as C0+; the only difference is peer access [repo: `salvage-fill-reserve-v2/PLAN.md`].

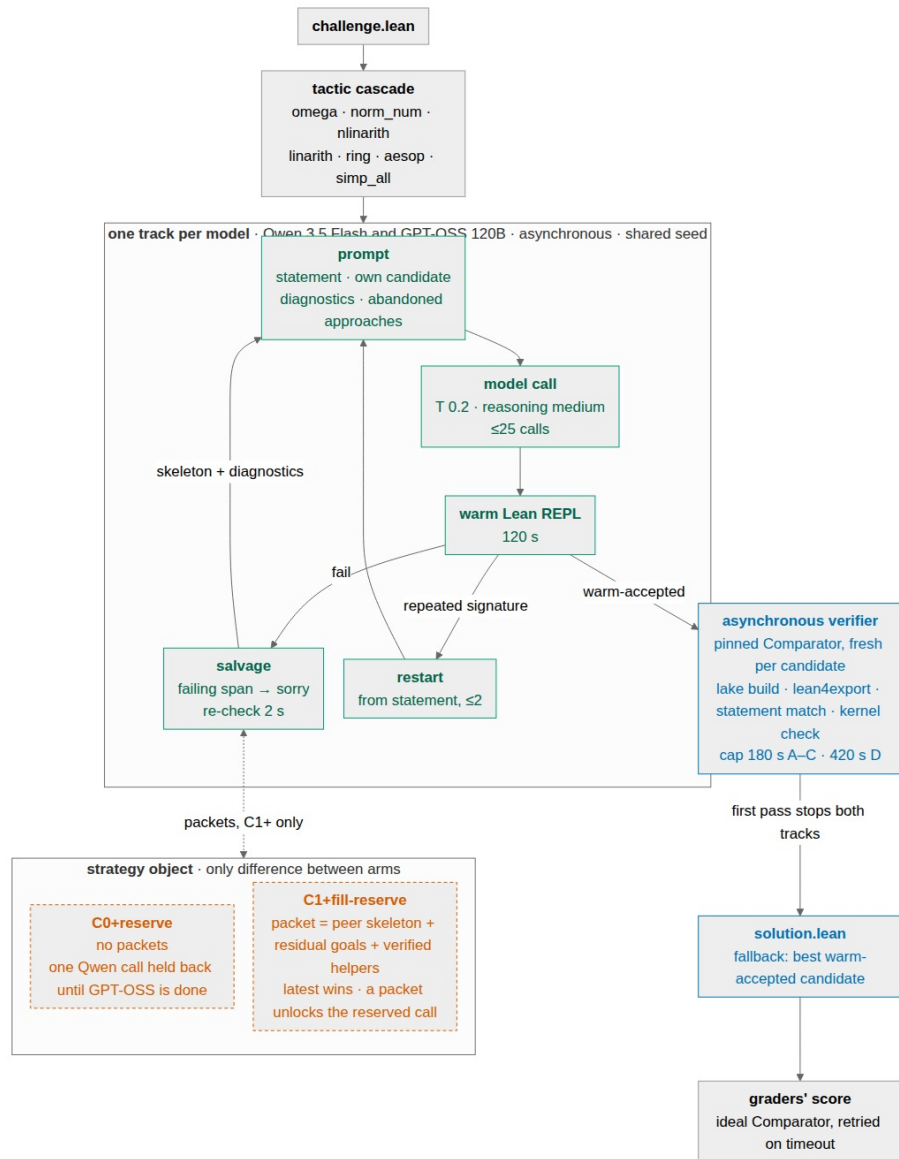


Figure 0a. Engine architecture. Bold labels are components; the strategy object is the only component that differs between arms.

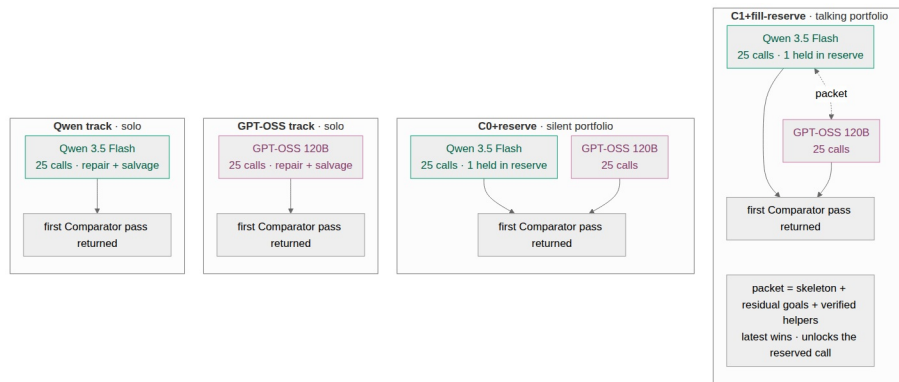


Figure 0b. The condition arms. Solo arms are one track of the same engine; C0+ adds the second track and a reserved Qwen call; C1+ adds peer packets and nothing else.

Why these, from the literature. Source-checked review in the repository (fable-5-1-report-online-development-v1.md). Control: independent sampling under a perfect verifier is the bar

any collaboration must clear (Brown et al.), so the silent portfolio. Salvage: largest gain the review found for off-the-shelf models on Lean (APOLLO: o4-mini 7.0 → 46.7% at 15 samples). Unit of exchange, a compiling skeleton with named holes: verified, so it cannot mislead; a few hundred characters against a 6,000-character failed file; fillable in place, so a hole type-checks where it sits or fails locally. Failed-candidate packets: nearest analog (arXiv 2608.18084) beats each model alone but credits 1.5% of solves to cross-model resampling, which is the C1/C2 result in advance. No critique in packets: unverified peer opinion loses to voting at matched compute and drifts to conformity in mixed-strength pairs (Debate-or-Vote, “Talk Isn’t Always Cheap”, Self-MoA). Next experiment (§8): sketch-and-fill with a filler, the one pattern the review found beating the union of two models (HybridProver, +15–18 points).

The submitted agent is C1+fill-reserve; family two is the one carried forward.

submission/agent.py promotes it under judge defaults (eight hours, \$1 ledger, 60 calls per model, dispatch cutoff twelve minutes before the deadline); scripts/judge_check.sh passes from a fresh clone of main. Why family two, independent of scores: small models. A failed-candidate packet is 6,000 characters of code known to be wrong plus “evaluate critically”. Anchoring on it is the failure the debate literature predicts; the one consumed packet in the ladder made its recipient worse (putnam_2020_a2, 22 diagnostics). A progress packet is the opposite object: short, verified, one objective (fill these holes). Plausibly helps a small model even below the detection floor of sixteen problems. Cost in calls, dollars, wall time, solved problems: none found. Benefit found: none.

The submitted commit (9333436) also carries the narrow-import fix of §7, byte-identical to the experiment code on the fourteen import Mathlib problems.

3. How it was measured

Sixteen problems, scored by the graders’ rule. All 16 supplied problems, two of them in upstream-corrected form (§7); defective and corrected versions are never pooled. The primary outcome is intention-to-treat pass/fail per cell under the graders’ stated procedure: an ideal Comparator on the returned solution.lean, raised timeout, retried on timeout, only a completed failing check scores zero. Every cell that failed in-cell while holding a warm-accepted candidate was re-graded on its returned file at 600 s; tables use that verdict and note the in-cell one where they differ. Cells that died before a real attempt count as failures, so every arm is scored on every block.

Matched blocks, paired contrasts. A cell is one problem attempted once by one condition. A block is one problem at one seed, a label shared by every condition attempting that problem together; “seed” here means a replication index, and “six seeds” means six matched attempts per problem. The seed was passed to the provider only in the later waves, and provider sampling is not fixed by it either way (§7). A preregistered six-problem core (p03, p06, p07, p10, putnam_2020_a2, rmo_2000_2) at seeds 1729, 2718, 3141; the other ten at one seed; 28 problem-seed blocks, each block’s conditions run concurrently on matched workers with rotating assignment. Per cell: \$1, 28-minute outer limit, 16-minute dispatch window, 120 s warm-Lean and 180 s Comparator timeouts (420 s in the final wave); uncertain-spend failures close the ledger and count as failures. Contrasts are paired within block.

A causal funnel, not a score, is the evidence for a mechanism. For every collaboration cell the log records whether each of six steps happened: a packet was produced; it was queued before the recipient ran out of calls; it was present in a request that completed; the next candidate reflected it; that candidate passed warm Lean; and it is in the returned proof on a block where the matched control failed. When all six hold, exchange caused the solve. When a talking arm outscores its control without a completed chain, the writeup reports the score difference and does not attribute it to the mechanism; §6.3’s p09 block is the worked example.

This design detects only large effects, and says so. For a two-arm comparison of per-problem solve probabilities at 80% power, band cells per arm scale as roughly $15.7 \cdot \tilde{p}(1-\tilde{p})/\delta^2$: at $\tilde{p} \approx 0.5$, $\delta = 0.5$ needs about 16, $\delta = 0.3$ about 45, $\delta = 0.2$ about 100. Three seeds on a six-problem band give about 16 band cells per arm: enough to detect a coin-flip problem becoming a near-certain solve; smaller effects need the final wave’s design. The final wave was designed against this arithmetic (§6.3). Matched seeds do not make the endpoints deterministic: byte-identical first requests returned different responses in matched cells [repo: STAGE2_SUBSTRATE_RERUN.md], so pairing controls time-of-day and infrastructure, not

sampling.

4. Does the two-model system beat either model alone?

By the union of the two plus what a second sample buys; on this set that is mostly Qwen twice. Three comparisons say the same thing at increasing resolution: the supplied baseline agent, the same-substrate single track at one seed, and the four-arm band study at six seeds.

4.1 Supplied baseline agent, one model at a time

Two runs per model of the kit’s single-model agent (25 turns, $T = 0.2$) [repo: outputs/baseline/]; per-problem table in Appendix E. Totals: Qwen 8/16 and 7/16, GPT-OSS 8/16 and 8/16; Qwen \$0.29–0.33 per set, GPT-OSS \$0.02–0.06.

The models are equal in aggregate, cheap enough that budget never binds, and fail on different problems, so the union of one run each is 10/16 (8/16 on the corrected set). Qwen costs five to fifteen times more per set (far more tokens) and is several times faster per call.

4.2 Same harness, one track at a time

On this engine Qwen’s track alone matches the two-model portfolio. The single-track arms (one model, communication off, salvage on, no reserve) ran on all sixteen problems at seed 1729, 32 cells [repo: evidence/archives/stage5-solo-20260903]. Qwen 9/16, GPT-OSS 6/16, union 9/16: everything GPT-OSS solved, Qwen did too, plus p07, p08, p10. The silent portfolio at the same seed is 9/16 (p09 solved, p10 lost to a two-call death); the talking portfolio 8/16.

one replication each	Qwen	GPT-OSS	union of solos	silent portfolio	talking portfolio
supplied baseline agent (refresh run)	7/16	6/16 †	8/16	9/16 ‡	-
this engine, single track (seed 1729)	9/16	6/16	9/16	9/16	8/16

One replication per cell; one-problem differences are within seed noise; the table maps which problems move. † Corrected-set passes only. ‡ The Aug 26 independent-repair-portfolio run: two stock repair loops, no communication.

Down the columns: harness engineering moved Qwen from 7 to 9 (p05 via the cascade, p10) and left GPT-OSS at 6 (gained p06, lost p07); the second, different track added nothing beyond Qwen alone; communication changed nothing again. The single-track comparison is half-compute (25 calls against the portfolio’s 2×25); the compute-matched control, a same-model portfolio, is reported in §4.3 as $1 - (1 - p)^2$ from the single-track rates, which is exact when the tracks do not interact (§7).

4.3 The band at six seeds, four arms

On 24 complete four-arm blocks (four band problems $\times$ six new seeds), the silent two-model portfolio solves 19; Qwen’s track alone 16; GPT-OSS’s 9; the union of the two solos 17. GPT-OSS’s track solved one block Qwen’s did not (p07, one seed). Per-problem solve rates on the 96 new-seed cells follow. The three “virtual” columns are compute-matched controls computed, not run: if a single Qwen track solves a problem with probability p , two independent Qwen tracks solve it with probability $1 - (1 - p)^2$, and the mixed virtual portfolio is $1 - (1 - p_{\text{Qwen}})(1 - p_{\text{GPT}})$. They answer “what would a 50-call portfolio of one model, or of the two models without any interaction, have scored?” (assumptions in §7):

six new seeds	Qwen	GPT-OSS	Qwen+Qwen (virtual)	GPT+GPT (virtual)	mixed (virtual)	silent portfolio	talking portfolio
p03_sq_ge_two_ab	6/6	6/6	1.00	1.00	1.00	6/6	6/6
p07_least_divisible	4/6	1/6	0.89	0.31	0.72	6/6	4/6
p08_sum_products	6/6	2/6	1.00	0.56	1.00	6/6	6/6

p09_imo1964	0/6	0/6	0.00	0.00	0.00	1/6	1/6
pooled rate	0.67	0.38	0.72	0.47	0.68	0.79	0.71

Pooled over the four problems, the silent portfolio exceeds the better solo by 0.13 (bootstrap 95% interval 0.00 to 0.25) and the virtual mixed portfolio by 0.11 (0.00 to 0.25); it also exceeds the virtual same-model Qwen+Qwen portfolio (0.72), though that control assumes independent tracks and §6.3 shows the models’ outcomes are positively correlated, so it is overstated. The whole gap is on p07 (6/6 against Qwen’s 4/6) and one p09 block. The reading that survives: a second 25-call track is worth about one extra band problem in ten, and on this set a second Qwen track would have been worth nearly as much as a GPT-OSS one. Figure 3 draws the four curves.

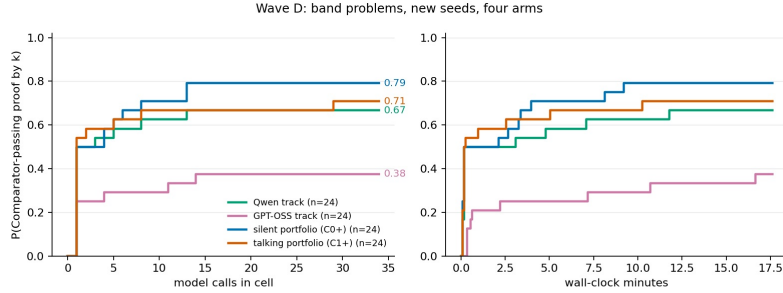


Figure 3. Wave D: the four arms on the band problems, new seeds only, by calls and by wall time.

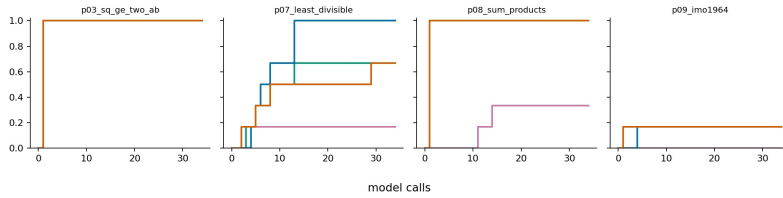


Figure 4. Wave D per-problem curves.

5. What does each model contribute?

Qwen searches; GPT-OSS structures; the portfolio takes Qwen’s proof because it arrives first. The returned proof records its track, so the division is visible directly. Across the 80 passing cells of the two matched waves, Qwen supplied 53, GPT-OSS 23, the cascade 4:

Problem	Qwen	GPT-OSS	cascade		Problem	Qwen	GPT-OSS	cascade
p01_linear	3	2	-		p06_pow_mod	8	7	-
p02_frac_cancel	2	3	-		p07_least_divisible	6	5	-
p03_sq_ge_two_ab	12	2	-		p08_sum_products	4	-	-
p04_sum_sq	3	2	-		p09_imo1964	2	-	-
p05_gcd_mersenne	-	1	4		p10_factorial_pow	13	1	-

Attribution is speed-biased (first accepted proof wins; Qwen is faster), so near-ties mean both tracks can solve those problems, while p08, p09, p10 are Qwen’s and p05 is GPT-OSS’s or the cascade’s. Qwen is several times faster per call and produces more distinct candidates per unit wall; it wins where the difficulty is the tactic sequence. GPT-OSS’s failures leave a compiling skeleton with real proof content 40% of the time against Qwen’s 11%, and it wins where a specific lemma or decomposition is needed (p05). Within a cell Qwen typically finishes its calls while GPT-OSS is on its second or third.

The harness contributes more than the second model. p10 is 0/2 under the supplied agent (both GPT-OSS runs died early with unknown cost; both Qwen runs went 25 turns and never built) and 14/14 across every arm of the two matched waves, thirteen from Qwen’s track. The difference is the repair loop: the supplied agent sends the model only the compiler messages from its last attempt, never the candidate they refer to, so every turn regenerates a file from error text about code it cannot see; the engine repairs the candidate in place, keeps

a bounded memory of abandoned approaches, restarts on a stalled error signature, and tries the tactic cascade on holes.

The complementarity is real and the portfolio captures it for free. Converting it into *interaction* needs GPT-OSS's structure to exist before Qwen runs out of calls, and Qwen's failures to contain something GPT-OSS can use, which the 11% salvage yield says they mostly do not. That is the question of \$6.

6. Does letting the tracks talk add anything?

No detectable score effect across three mechanisms, two matched waves, and a powered band study; a request-level replay showing the packet does not change the next response; and a structural reason.

6.1 Wave A: failed-candidate packets, 84 cells, commit 2e15754 (pilot-range evidence; detail in Appendix E)

C0 15/28, C1 16/28, C2 16/28; curves within one cell at every budget. All 84 cells completed [repo: evidence/online-development-v1-stage3-2e157547]; re-grading changed nothing. Five discordant blocks, none an arm-level pattern except p07 (C0 1/3, C1 3/3, C2 2/3), and none of C1's three p07 proofs came from a packet-exposed attempt. Across the 55 C1 and C2 cells, 309 attempts had a packet in context, 25 were warm-accepted, nine were the returned proof, seven of those on blocks C0 also solved; the one cell in the project where the only passing arm had a packet in context on the accepted attempt (p09, C2) is Qwen repairing its own attempt-5 candidate with no declaration, lemma, or structure from the packet (Appendix A.1).

Family one was dropped on priors, because the pilot could not give a read. Three seeds on 28 blocks cannot separate arms one cell apart (\$3), and the delivery log adds that the treatment mostly never reached its recipient (\$6.4), so the numbers above neither support nor condemn the mechanism. With no read from the data, the choice went to the family with the stronger priors. Family two's within-track half, salvage, has the strongest direct evidence the review turned up for off-the-shelf models, with the usual caveat that reviews miss things (APOLLO: o4-mini 7.0 → 46.7% at 15 samples; Cobblestone; DSP+ line masking), and its exchange half is the only pattern in that review that beat the union of two models (HybridProver). Family one's priors are negative: the nearest analog credits 1.5% of its solves to cross-model resampling, and unverified critique loses to sampling at matched compute in every debate study without a verifier. That decision rule would have been right with the pilot numbers reversed.

6.2 Wave C: progress packets with a reserved call, 56 matched cells, commit e8af4fb

Between the waves, a fourteen-cell salvage ladder (Appendix D) produced the first legible funnel (a packet-exposed GPT-OSS response yielding new compiling state on p06) and one consumed packet that made its recipient worse (putnam_2020_a2, 22 diagnostics); it changed the receiving contract to fill-the-holes and added the reserved call.

C0+reserve 17/28, C1+fill-reserve 16/28; the curves lie on top of each other. Same 28 blocks as Wave A [repo: stage3-matched-v1/], graders' rule (\$3): both p08 cells pass on re-grade, both rmo_2000_6 cells are completed rejections (\$7), two early deaths score on their returned file (p02 C1+ pass, p10 C0+ fail). Discordant: p07 seed 1729 and p09 (C0+ ✓), p10 seed 1729 (C1+ ✓). Spend, calls, and wall time match within noise. Per-problem table in Appendix E.

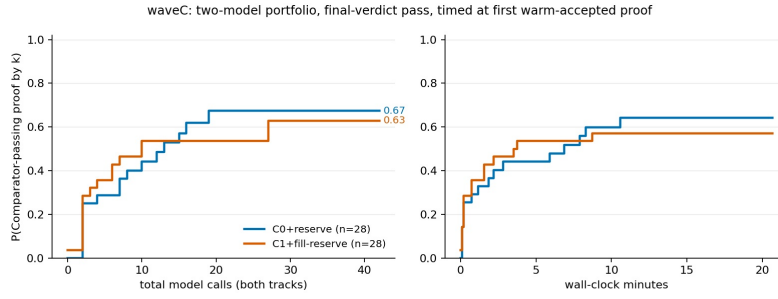


Figure 1. Wave C portfolio search curves by calls and by wall time.

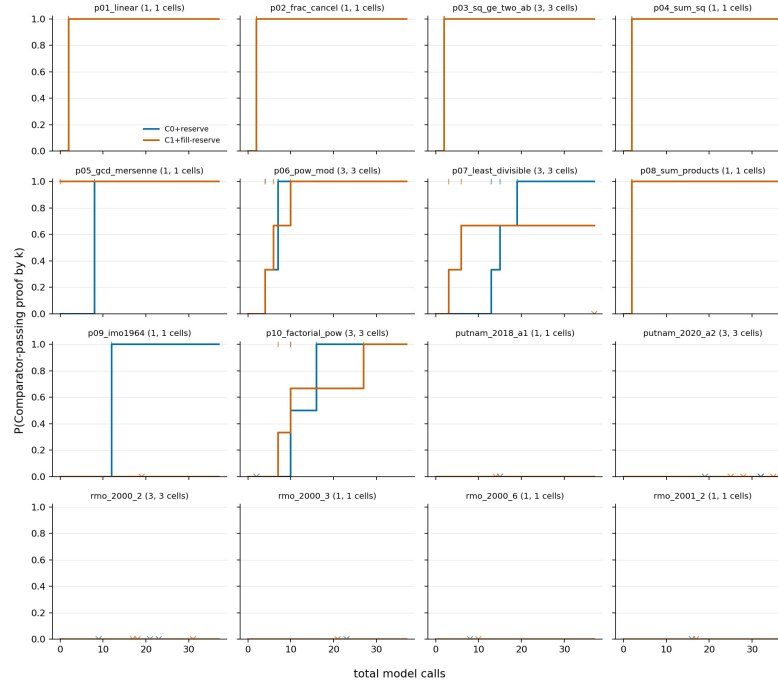


Figure 2. Wave C per-problem search curves. Ticks mark passing cells at their first accepted proof; $\times$ a censored (failed) cell.

C1+ reaches its accepted proofs earlier in calls on the problems it solves (median 3.5 vs 7), which the reserved call was meant to buy, and converts it into no additional problem. The floor set is unchanged: putnam_2018_a1, putnam_2020_a2, rmo_2000_2, rmo_2000_3, rmo_2001_2, 0 in 22 cells.

The mechanism now works end to end and still does not move the score. Twenty-five peer_fill calls ran in 13 of 27 C1+ cells, so the reserve fired; nine packet-exposed responses were warm-accepted, and an offline recheck of ten recovered packet-exposed candidates found all ten pass the Comparator [repo: packet-candidate-recheck-20260902T1600Z]. Two were the proof the cell returned (p06 seed 2718, GPT-OSS attempt 5; p07 seed 3141, Qwen attempt 15). In the p06 cell the packet-exposed response was the cell's first accepted proof: GPT-OSS had been failing on maxRecDepth, Qwen's skeleton carried the $7^4 \equiv 1 \pmod{100}$ structure with one hole, GPT-OSS filled it with norm_num (Appendix A.5). These are the first trajectories in which compiler-validated peer material is produced, delivered, consumed, and yields a passing proof. They are not uplift: the matched C0+ cells solved both problems (p06 at call 4, faster), and the p06 "hole" was already closed. The other seven passing packet-exposed candidates sat on blocks where the receiving track's peer already had a proof.

6.2b Counterfactual replay: the same request with and without the packet

At eight samples per variant, a packet in context does not change the chance that the recipient's next response compiles: -0.01 for progress packets, $+0.00$ for failed-candidate packets. Design: every request whose prompt contained a packet block was re-issued from the recorded state with the packet block present and with it cut (the cut removes

the packet and its instruction line, leaving the prompt the model would have seen had no packet been produced); same model, seed, temperature, and reasoning setting; every response warm-Lean checked. Unit of analysis is the source request; intervals are request-level bootstrap. Three passes, 2,066 responses, \$2.91 in model spend, five errors [repo: experiments/analysis/packet-replay*.csv; scripts/run_packet_replay.py].

packet type	source requests	samples per variant	compiles with packet	without	difference (95% interval)	requests up / down
progress (Wave C)	25	8	0.375	0.380	−0.01 (−0.10, +0.10)	4 / 7
progress (Wave D, band problems, new seeds)	22	8	0.43	0.44	−0.01 (−0.05, +0.05)	3 / 4
failed-candidate (Wave A, all 304)	304	2	0.086	0.081	+0.00 (−0.02, +0.02)	12 / 10
progress (Wave C), first pass	25	2	0.40	0.26	+0.14 (+0.04, +0.26)	5 / 0

The first pass was noise. The two-sample pilot (last row) read as a 14-point gain with five requests up and none down. Re-sampling the same 25 requests at eight per variant put four up, seven down, and the difference at −0.01; twenty-two requests from Wave D’s talking cells, on the band problems where an effect would matter, gave the same answer with a tighter interval. Two samples resolve a request’s rate to 0, ½, or 1, and with 25 requests that resolution produced a two-sided sign-test p of 0.06 from what turned out to be sampling variation. The number that survives is the eight-sample one.

One block is real and already explained. The p06 seed 2718 pair (Appendix A.5) holds at eight samples: +0.38 in both directions (Qwen’s skeleton into GPT-OSS: 8/8 against 5/8; GPT-OSS’s back into Qwen: 3/8 against 0/8). It is the case in §6.4 where the packet is a near-complete proof with one hole that norm_num closes; the silent twin solved the block at call 4. On p07 the packet hurts (0.16 against 0.41 over four Wave C requests; 0.04 against 0.07 over twelve Wave D requests): the skeleton from a model that has the wrong decomposition anchors the receiver on it.

Long prefixes hurt, short ones help; the average is zero. Every progress packet in the study has one hole: salvage keeps the longest compiling prefix of the peer’s candidate and closes it with all_goals sorry, so a “skeleton” is the peer’s first k compiling lines, not a decomposition. Across the 25 Wave C requests the with-minus-without difference falls with k (correlation −0.45): the three most negative requests carry the longest prefixes (p07, 14 lines, −0.63 and −0.38; p10, 19 lines, −0.25), the positive ones carry one to seven lines (p01 +0.75, p06 +0.38 twice). Twenty-five requests, confounded with problem, so this is a direction rather than an estimate; but it is the sign structure a null of exactly zero needs. A long prefix from a model that is stuck is a wrong plan the receiver inherits; a short one is a verified lemma or two it can use or ignore.

What the replay settles. The score null of §6.2 and §6.3 is not a delivery or scheduling artefact and not a power artefact at the request level: with the packet in the exact recorded context, sampled eight times, the recipient compiles at the same rate as without it. Whatever a compiler-grounded packet carries, these two models do not convert it into a compiling next response except when the packet is already a proof. Limits: warm Lean rather than the Comparator; the next response only, not the rest of the trajectory; conditional on a packet having been produced.

6.3 Wave D: the powered band study, four arms, commit d43af01

Seeds spent only where the outcome can move. A problem is in the band if, across every portfolio cell in Waves A and C, it has at least one pass and one fail; applied before launch, that admits p03, p07, p08, p09 and excludes six problems every cell solved, five no cell solved, and rmo_2000_6 (unscorable on the experiment commit, §7). Six new seeds, four arms on

identical blocks (each model as a single track; C0+reserve; C1+fill-reserve), 96 cells, blocks dispatched with all four arms adjacent in a fixed round-robin; settings as Wave C except the 420 s Comparator cap; stopping rule in `plan.json`, intention-to-treat on every cell completed at analysis, pilot seeds not pooled. With six seeds the band contrasts carry roughly ± 0.4 problems of uncertainty on a 16-problem scale.

This is the decisive comparison of the writeup. On the same problems, with the same six replications each, the talking portfolio (C1+) solved 17 of 24 cells and the silent portfolio (C0+) solved 19 of 24. All 96 cells completed; every block has all four arms. Pooled solve rate 0.71 against 0.79.

six new replications	silent portfolio (C0+)	talking portfolio (C1+)	difference
p03_sq_ge_two_ab	6/6	6/6	0
p07_least_divisible	6/6	4/6	-2
p08_sum_products	6/6	6/6	0
p09_imo1964	1/6	1/6	0
total	19/24	17/24	-2

The pooled contrast, talking minus silent, is -0.08 with a bootstrap 95% interval of -0.25 to $+0.04$; this sample rules out an uplift from talking larger than about 4 points on these problems and is consistent with a small cost. Four problem-replication pairs disagree between the arms. p09 seed 4481 passed under C1+ and failed under C0+: Qwen’s track solved it on its first, pre-packet attempt with no packet ever produced in the cell, which is provider nondeterminism on identical first requests, not the mechanism. p07 seeds 4481 and 5279 and p09 seed 6211 passed under C0+ and failed under C1+. The funnel is empty at the end: nineteen packet-exposed attempts across the 24 C1+ cells, none warm-accepted, none returned. Counting the opportunity set directly on the 24 blocks: both solo tracks fail on 7, and there the silent portfolio solves 2 and the talking portfolio 1 (the p09 case above); exactly one solo solves on 9, silent 9/9, talking 8/9; both solve on 8, both portfolios 8/8. The portfolio recovers the union and a little more (a second sample of Qwen on p07); exchange had seven blocks to act on and did not beat the control on them. The solo union (0.71) falls short of the independence prediction (0.79), so the models’ failures are positively correlated, and the virtual same-model control of \$4.3, which assumes independence, overstates the same-model portfolio: that comparison is conservative for the two-model system.

6.4 Why not: timing, content, noise, and a ceiling

Timing was fixed and did not matter. Under latency skew, “reciprocal” exchange is asymmetric: in the independent-scheduler canary C1 delivered one packet to GPT-OSS and left the other queued for a Qwen track that had already finished; C2 left three of four queued. The reserve in Wave C fixed delivery, with packets consumed on the reserved call in 13 of 27 cells, and the score did not move.

Content that crosses is either useless or already a proof. Failed-candidate packets, once consumed, trace to no accepted proof. Compiler-grounded packets are legible but rare at small ceilings; on `putnam_2020_a2` the one consumed made its recipient worse (22 diagnostics); in Wave C every packet-exposed passing proof sat on a block the silent twin also solved, and in the one trajectory where the packet was decisive for its receiver the skeleton was already complete. A skeleton from a model that lacks the right Mathlib names is not obviously better than a blank slate; one that already solves the problem is a portfolio win.

Noise flips single cells. Provider nondeterminism at fixed seed is enough to flip a cell, so a one- or two-cell lead on sixteen problems is below the detection floor; every gate therefore asked for a trace, not a score.

The ceiling is structural. Exchange can only add on blocks where *both* tracks fail alone. Here the both-fail set is the floor set (five problems, zero in 140 cells) plus the occasional band block at a bad seed, and on the floor set neither model produces structure the other can use (48 calls on `putnam_2020_a2`: six skeletons, no progress). The complementarity that exists is the kind a portfolio captures for free. In the powered study that leaves seven of 24 blocks for the mechanism to act on, and on those the talking arm scored 1 against the silent arm’s 2. Published positive results for exchange come from models that make different errors on answerable problems; these two share a bottleneck, Lean-level lemma and tactic knowledge.

7. What is not controlled

Provider nondeterminism. Identical requests return different completions; pairing controls time-of-day and infrastructure, not sampling. Contrasts are paired-descriptive.

Substrate changes during development. Gate, scheduler, and verification were repaired mid-project; results are never pooled across a repair. The p10 change between the pre-audit wave and Wave A is a repair. The in-agent Comparator cap (180 s in A-C, 420 s in D) affects only whether a confirmed proof stops the search, never the verdict.

Reasoning setting. The supplied-agent solos in §4.1 sent no reasoning field (provider default); every engine run from Wave A on sent explicit medium. That is one more reason §4.1 and the engine rows are not the same substrate, and why §4.2 and §4.3 carry the solo comparison.

Compute matching. The single-track control has 25 calls against the portfolio's 2×25 . The compute-matched control is not a 50-turn chain, because repair chains saturate (the restart rule exists for that reason); it is two independent tracks of the same model, computable as $1 - (1 - p)^2$ from the single-track rate because silent tracks do not interact. That assumes uncorrelated outcomes given the problem and slightly favours the virtual portfolio, whose tracks do not share a box.

Latency. GPT-OSS latency varied fourfold across days; wall-time comparisons across waves are not meaningful.

Problem set. Sixteen problems, six solved by everything, five by nothing. The effective sample is the band.

Import canonicalization. `rmo_2000_3` and `rmo_2000_6` declare narrow imports; the agent's `import Mathlib` canonicalization made every warm-accepted `rmo_2000_6` proof a certain statement mismatch at the Comparator, so those cells are unscorable on the experiment commit. The kit's own REPL has the same disagreement with the Comparator for any agent. Fix in the submitted commit; detail, validation, and the two upstream corrections in Appendix H.

Infrastructure. Lost or zero-call cells are documented per wave and never pooled with outcomes.

Analyst degrees of freedom. Core/breadth split, seeds, and contrasts were preregistered before each launch; salvage gates were written before each stage; Wave D's band rule and seeds were in its plan before launch and its estimate uses only new seeds. Not preregistered: which mechanism to try next.

8. Next steps

Next: sketch-and-fill on a wider set. The design the data point at is role-asymmetric: GPT-OSS writes a skeleton with explicit holes first, Qwen fills, GPT-OSS gets the residual goals back; paired against same-budget C0+ and a same-model portfolio, on a set with a real middle band, at five or more seeds. The replay of §6.2b runs on its packets at eight samples per variant before any score is read.

Widen the problem set, mechanically. The first 32 miniF2F test statements that build against the pinned Mathlib with the `import Mathlib` header and are not closed by the tactic cascade, in benchmark order, no hand-picking; a Qwen single-track screen at eight attempts per problem (10-call cap) first on the 16 known problems to calibrate the screen against the observed band, then on the 32; band = 1 to 7 of 8; then the four arms on the band at four or more replications. In progress at the time of writing (`experiments/stage6-expanded/`).

Measure complementarity directly. Solo runs on the new set give the both-fail set, which bounds what any exchange mechanism can add; on a set where the two models' failures diverge, the same harness and funnel test the mechanism where it can act.

Change what the packet carries, then rerun the replay. The prefix-length pattern in §6.2b points at the packet builder: send the verified have lemmas only, or cap the prefix at a few lines, and re-run the eight-sample replay on the same 47 recorded states for about \$2 before any wave. Separately, salvage yields a usable skeleton 24% of the time (11% for Qwen); salvaging at the tactic-block level rather than the smallest failing span is the change

that raises it. The counterfactual replay (\$6.2b) is the cheapest instrument in the study (\$3 for 2,000 responses) and runs on every new wave’s packets at eight samples per variant; a two-sample version should not be reported.

Appendix A. Transcript evidence

Paths are on the evidence branches.

1. **p09_imo1964, Wave A, C2** (\$6.1): evidence/online-development-v1-stage3-2e157547, task stage3v1-breadth-p09_imo1964-r1-c2, transcript.json calls 6 (Qwen attempt 5, pre-packet candidate with pow_two_mod_7) and 8 (attempt 6, peer_fill, accepted). The GPT-OSS Nat.ModEq packet is in the attempt-6 request; the accepted file contains none of its declarations.
2. **Independent-scheduler canary** (\$6.4): experiments/online-development-v1/stage2-independent-scheduler-canary-r1-results.json; stage2async1-* transcripts on evidence/all-transcripts-20260902.
3. **Salvage Stage 2 microcell** (Appendix D): experiments/salvage-progress-packets-v1/stage2-microcell/artifacts/worker9-clplus/, Qwen call 4 (22-line skeleton), GPT call 2 (packet consumed), GPT salvage (6-line skeleton).
4. **Salvage Stage 3, seed 3141 C1+** (\$6.4): experiments/salvage-progress-packets-v1/stage3-putnam-a2/tasks/salvage-stage3-putnam2020a2-r3-clplus/, Qwen call 3.
5. **p06_pow_mod seed 2718, Wave C, C1+** (\$6.2): evidence/run-data-e8af4fb7d278, task stage3fill-core-p06_pow_mod-r2-clplus-fill-reserve, transcript.json call 10 (GPT-OSS attempt 5, peer_fill): the request carries Qwen’s skeleton (h1 : $7^4 \% 100 = 1$, h2 : $2026 = 4 * 506 + 2$, all_goals sorry); the response replaces sorry with norm_num; the next request reports warm acceptance; recheck record packet-candidate-recheck-20260902T1600Z/.../f4ce2f99.../result.json shows the Comparator pass.
6. **Byte-identical requests, divergent responses** (\$7): experiments/salvage-progress-packets-v1/STAGE2_SUBSTRATE_RERUN.md; stage3-putnam-a2/RESULTS.md.
7. **Wave D, p09 seed 4481, C1+** (\$6.3): evidence/archives/stage5-band-20260903/hosts/takehome-worker-5/.../stage5-band-p09_imo1964-s4481-clplus-fill-reserve/, Qwen attempt 1 (direct, no packet in the cell) accepted; the matched C0+ cell’s identical first request failed.

Appendix B. Resource accounting

Spend is OpenRouter’s returned usage.cost per cell ledger. Wave A: 84 cells, \$2.26 (\$0.88 / \$0.66 / \$0.72), one sub-two-minute death (rmo_2000_6 C1), two cost_unknown at the outer limit (rmo_2000_3 C1, C2), no final-Comparator timeouts. Wave C: 56 cells, \$1.86 (\$0.94 / \$0.92), two sub-ten-second deaths, two final-Comparator timeouts (p08, re-graded), max per-cell \$0.063. Salvage ladder: 14 cells, under \$0.20. Supplied-agent solos: four runs, \$0.70. Same-harness single tracks: 32 cells, \$0.49. Wave D: 100 cells (96 new-seed, 4 pilot), \$1.94. Counterfactual replay: 2,066 responses, \$2.91. Earlier development waves are inventoried in experiments/online-development-v1/EXPERIMENT_ARCHIVE_INDEX.md; no cell exceeded its \$1 cap.

Appendix C. Tools disclosure

The harness, orchestration, and most code were written with OpenAI Codex on the experiment workers under my direction and review; the collaboration design, conditions, stage gates, and promotion decisions are mine. Analysis of the archives, the figures, and drafting of this writeup were done with Claude (Anthropic) with read access to the repository and evidence branches; it also reviewed the engine independently on Aug 28. I checked every number here against the artifact it cites.

Appendix D. The salvage ladder (Wave B)

Stage	Problem, seed	Calls/model	C0+	C1+	What it established
					4 packets produced, 1 consumed; the packet-exposed

2 (first)	p06, 1729	4	✓ (5 calls)	✗ (8 calls)	GPT call produced a new compiling 6-line skeleton. Exposed the synchronous-Comparator stall.
2 (rerun)	p06, 1729	4	invalid	✓	Ranking bug: a rejected call-zero candidate outranked a warm-valid GPT proof. Fixed, regression test added.
2 (final)	p06, 1729	4	✗ (8)	✓ (8)	Not uplift: the winning Qwen call used no packet; the one packet was never consumed; identical first requests diverged.
3	putnam_2020_a2 × 3 seeds	4	0/3	0/3	Six skeletons in 48 calls (4 C0+, 2 C1+). Only seed 3141 C1+ produced packets (2, 1 consumed); the exposed response produced 22 diagnostics, no progress.

[repo: salvage-progress-packets-v1/STAGE*_RESULTS.md]

Appendix E. Per-problem tables

Supplied baseline agent, one model at a time (§4.1).

Problem	Qwen (Aug 23)	Qwen (refresh)	GPT-OSS (Aug 23)	GPT-OSS (refresh)
p01_linear	✓	✓	✓	✓
p02_frac_cancel	✓	✓	✓	✓
p03_sq_ge_two_ab	✓	✓	✓	✓
p04_sum_sq	✓	✓	✓	✓
p05_gcd_mersenne	✗	✗	✓	✓
p06_pow_mod	✓	✓	✓	✗
p07_least_divisible	✓	✓	✓	✓
p08_sum_products	✓	✓	✗	✗
p09_imo1964	✓	✗	✗	✗
p10_factorial_pow	✗	✗	✗	✗
putnam_2018_a1 †	✗	✗	✗	✓
putnam_2020_a2 †	✗	✗	✓	✓
rmo_2000_2	✗	✗	✗	✗
rmo_2000_3	✗	✗	✗	✗
rmo_2000_6 ‡	✗	✗	✗	✗
rmo_2001_2	✗	✗	✗	✗
Total	8/16	7/16	8/16	8/16
Cost	\$0.285	\$0.331	\$0.019	\$0.062

† Pre-correction passes (the answer object could be redefined); 0/22 and 0/8 on the corrected set. ‡ Pre-correction statement was false.

Wave C, C0+reserve against C1+fill-reserve (§6.2).

Problem	C0+reserve	C1+fill-reserve		Problem	C0+reserve	C1+fill-reserve
p01_linear	1/1	1/1		p09_imo1964	1/1	0/1
p02_frac_cancel	1/1	1/1 †		p10_factorial_pow	2/3 †	3/3
p03_sq_ge_two_ab	3/3	3/3		putnam_2018_a1	0/1	0/1
p04_sum_sq	1/1	1/1		putnam_2020_a2	0/3	0/3
p05_gcd_mersenne	1/1	1/1 ‡		rmo_2000_2	0/3	0/3
p06_pow_mod	3/3	3/3		rmo_2000_3	0/1 §	0/1 §
p07_least_divisible	3/3	2/3		rmo_2000_6	0/1 §	0/1 §
p08_sum_products	1/1 ¶	1/1 ¶		rmo_2001_2	0/1	0/1
Total	17/28	16/28				

† Died after two calls; scored on its returned file. ‡ Closed by the tactic cascade (in the C0+cell the cascade missed it and GPT-OSS solved it at attempt 8). § Narrow-import problems, §7. ¶ In-cell verdict was a 181 s Comparator timeout in both arms; re-graded pass.

Wave A detail (family one, §6.1).

Per-problem results for the three family-one arms on the 28 matched blocks. Communication arms used fewer calls (median 19 vs 11 and 10) and dollars (\$0.88 vs \$0.66 and \$0.72) because this wave’s verifier stopped search on a confirmed proof and failed cells in every arm ran to the cutoff; that is the stopping rule, not a benefit. Discordant blocks: p07 seeds 1729 and 3141 (C0 ✗, C1 ✓, C2 split), p03 seed 3141 (C2 ✗), p08 (C1 ✗), p09 (only C2 ✓). p10 is 9/9 here against 5/9 in the pre-audit version of the same design; that is the gate and scheduler repair, present in every arm.

Problem	C0	C1	C2		Problem	C0	C1	C2
p01_linear	1/1	1/1	1/1		p09_imo1964	0/1	0/1	1/1
p02_frac_cancel	1/1	1/1	1/1		p10_factorial_pow	3/3	3/3	3/3
p03_sq_ge_two_ab	3/3	3/3	2/3		putnam_2018_a1	0/1	0/1	0/1
p04_sum_sq	1/1	1/1	1/1		putnam_2020_a2	0/3	0/3	0/3
p05_gcd_mersenne	1/1	1/1	1/1		rmo_2000_2	0/3	0/3	0/3
p06_pow_mod	3/3	3/3	3/3		rmo_2000_3	0/1	0/1	0/1
p07_least_divisible	1/3	3/3	2/3		rmo_2000_6	0/1	0/1	0/1
p08_sum_products	1/1	0/1	1/1		rmo_2001_2	0/1	0/1	0/1
Total	15/28	16/28	16/28					

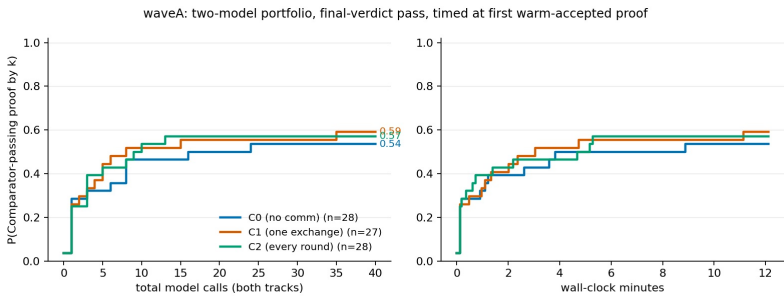


Figure E1. Wave A portfolio search curves: probability that a cell ends with a Comparator-passing proof, placed at the call (left) or minute (right) of its first warm-accepted candidate; failed cells counted as never solved.

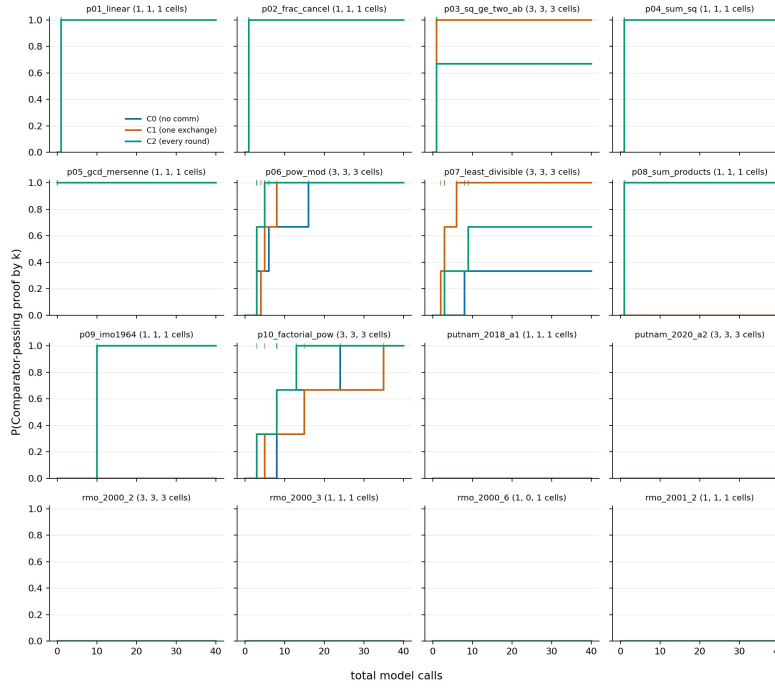


Figure E2. Wave A per-problem search curves.

All three curves rise together over the first five calls, where the saturated problems are solved, then separate by at most one cell.

Appendix F. Comparator cost

Resource numbers in this wave measure the dispatch window, not search. The in-agent verifier started 87 Comparator runs; 64 hit the 180 s cap, 19 passed, 4 failed; the final check on the same box took median 118 s (p90 166, max 181), so the cap was too tight. In 32 of 33 cells that reached a warm-accepted proof, dispatch continued afterward, and 10 passing cells passed through the fallback checkpoint rather than a live verified success. Verdicts and search curves are unaffected once the p08 cells are re-graded; this only ever cost time-to-information, and the final wave raised the cap to 420 s.

Each Comparator run builds and exports the Challenge and the Solution from scratch in a fresh container (two lake builds, two lean4exports, a kernel replay), with nothing cached between runs of the same problem. Observed on the 8 GB droplets:

Where	Observed duration	Setting
Concurrency canary (Aug 30)	90-115 s	one run at a time
Stage 1 boundary, worker 9	148 s	idle box
Salvage Stage 2 microcells	165-183 s per intermediate check	agent + warm REPL active
Wave C, final end-of-cell check	median 118 s, p90 166 s, max 181 s	agent finished
Wave C, in-agent checks	64 of 87 exceeded 180 s	agent + warm REPL active
Re-grade of the two p08 files, worker 9	~670 s each	box under other load
Wave D, first in-agent check (p03)	206 s	420 s cap, agent + REPL active

This per-run cost is what the asynchronous verifier, the fallback ranking, the 180 → 420 s change, and the re-grade protocol respond to; verdicts were never wrong when they completed. Building and exporting the Challenge once per problem and reusing a warm container for Solutions would take a check from minutes to seconds on the same hardware, and is the one kit change I would propose upstream.

Appendix H. Narrow-import problems and upstream corrections

rmo_2000_3 and rmo_2000_6 declare narrow imports. The agent canonicalizes every candidate to

import Mathlib and checks it in a full-Mathlib REPL; all four rmo_2000_6 cells holding a warm-accepted proof were rejected with “theorem statement do not match” although the statement text is byte-identical, because under full Mathlib $a^2 : \mathbb{N}$ elaborates through a different instance and the exported kernel terms differ. The kit’s REPL strips imports and checks under import Mathlib, so its feedback and the Comparator disagree on these problems for any agent; mine made the mismatch certain. The fix (candidates and the warm REPL keep the challenge’s pristine import block) is in the submitted commit; validated on one live rmo_2000_6 run (statement matched, proof failed on an unsolved goal in 21 s); the 24-cell rmo_2000_6 tail queued behind Wave D had not started at the deadline. Upstream corrections: rmo_2000_6 second minimum $20 \rightarrow 10$ after I sent a Lean disproof; Putnam answer sets inlined against circular proofs.

Appendix G. References

Brown et al., “Large Language Monkeys: Scaling Inference Compute with Repeated Sampling”, arXiv 2407.21787. Ospanov et al., “APOLLO: Automated LLM and Lean Collaboration for Advanced Formal Reasoning”, arXiv 2505.05758. “Compiler-Guided Adaptive Proof Search with Cross-Model Synergy on Context-Dependent Theorem Proving”, arXiv 2608.18084. “Debate or Vote: Which Way Goes the Wisdom of LLM Crowds?”, arXiv 2508.17536. “Talk Isn’t Always Cheap: Understanding Failure Modes in Multi-Agent Debate”, arXiv 2509.05396. Li et al., “Rethinking Mixture-of-Agents: Is Mixing Different LLMs Beneficial?” (Self-MoA), arXiv 2502.00674. “HybridProver: Augmenting Theorem Proving with LLM-Driven Proof Synthesis and Refinement”, arXiv 2505.15740. Sanchez-Stern et al., “Cobblestone: Iterative Automation for Formal Verification”, arXiv 2410.19940. “DSP+: LLM-based Proof Generation with Fine-grained Draft, Sketch and Prove”, arXiv 2506.11487. Full evidence map with per-paper effects and matching status: fable-5-1-report-online-development-v1.md in the repository.